Projet individuel – Architecture de données SmartCampus

TD 4 – Paradigmes NoSQL et benchmark

Durée : 1 h 30

Technologies et performances

1. Contexte et problématique

SmartCampus doit traiter des données provenant de plusieurs centaines de capteurs.

Ces équipements transmettent régulièrement des informations qui doivent être stockées, consultées et analysées.

La plateforme doit également permettre de consulter les caractéristiques des équipements, de retrouver leur dernier état et d'explorer leurs dépendances techniques.

Une base relationnelle peut couvrir une partie importante de ces besoins, mais certaines fonctionnalités pourraient bénéficier de modèles de stockage différents.

Votre mission consiste à étudier les quatre paradigmes NoSQL et à déterminer leur pertinence pour les usages de SmartCampus.

Vous devez ensuite proposer un protocole de benchmark permettant de comparer une solution relationnelle à une solution NoSQL adaptée.

Problématique :

Comment sélectionner un paradigme de stockage en fonction des usages et comment démontrer objectivement son intérêt technique ?

2. Travail demandé

Étape 1 – Associer chaque scénario à un paradigme

Vous devez étudier les quatre scénarios suivants.

Scénario A – Fiche équipement complète

Un technicien consulte une fiche détaillée comprenant les caractéristiques d'un équipement, sa configuration, ses capteurs associés et son état actuel.

Paradigme étudié : base orientée documents.

Travail demandé :

1. Identifiez les informations qui peuvent être regroupées dans un même document.
2. Déterminez quelles informations doivent être intégrées sous forme d'objets imbriqués ou de tableaux.
3. Identifiez les données qui doivent rester dans des collections distinctes.
4. Expliquez les avantages de cette organisation pour une consultation complète.
5. Analysez les risques liés à la duplication des données.

Questions de réflexion :

- Faut-il embarquer les dernières mesures dans le document de l'équipement ?
- Faut-il embarquer l'ensemble des interventions historiques ?
- Comment mettre à jour les informations d'un équipement ?
- Que se passe-t-il lorsqu'un capteur est déplacé ?

Attention : un document doit être conçu selon les besoins de consultation et non comme une copie systématique de toutes les données relationnelles.

Scénario B – Dernière valeur d'un capteur

Un tableau de bord doit afficher rapidement la dernière valeur connue de chaque capteur.

Les données sont fréquemment actualisées et les utilisateurs consultent régulièrement les états courants.

Paradigme étudié : base clé-valeur.

Travail demandé :

1. Proposez une convention de nommage des clés.
2. Déterminez la structure de la valeur stockée.
3. Définissez la stratégie de mise à jour.
4. Déterminez si une expiration automatique est nécessaire.
5. Expliquez comment éviter qu'une ancienne mesure remplace une mesure plus récente.

Exemple de clé :

```
sensor:CAP-001:last_state
```

Exemple de valeur :

```
{
  "value": 21.5,
  "unit": "C",
  "timestamp": "2026-11-10T08:30:00Z"
}
```

Questions de réflexion :

- Pourquoi utiliser une clé composée d'un préfixe et d'un identifiant ?
- Une expiration automatique est-elle toujours pertinente ?
- Quelle durée de validité retenir pour un état de capteur ?
- Comment gérer un capteur qui cesse de transmettre ?
- Comment reconstituer l'état courant si le cache est vidé ?

Vous devez distinguer l'état courant du capteur et son historique complet.

Scénario C – Historique des mesures

Les capteurs transmettent des mesures toutes les dix secondes.

L'application doit conserver ces informations et permettre de retrouver rapidement les mesures d'un capteur sur une période donnée.

Les requêtes peuvent porter sur :

- Les dernières mesures d'un capteur.
- L'historique d'un capteur sur 24 heures.
- Les mesures de tous les capteurs d'un bâtiment sur une journée.
- Les moyennes horaires ou journalières.

Paradigme étudié : base orientée colonnes.

Travail demandé :

1. Déterminez les attributs nécessaires au stockage.
2. Proposez une clé de partition.
3. Définissez les critères de tri des mesures.
4. Expliquez comment éviter des partitions trop volumineuses.
5. Identifiez les requêtes que votre modèle permet d'exécuter efficacement.
6. Définissez une stratégie de rétention des données.

Exemple de structure conceptuelle :

```
MESURES

- id_capteur

- date_mesure

- valeur

- unite
```

Vous pouvez étudier une clé de partition construite à partir de l'identifiant du capteur et d'une période temporelle.

Par exemple :

```
Partition : CAP-001 / 2026-11-10

Clustering : date_mesure
```

Questions de réflexion :

- Pourquoi ne pas utiliser uniquement la date comme clé de partition ?
- Comment répartir les données entre plusieurs nœuds ?
- Comment gérer un volume croissant de mesures ?
- Quel impact le choix de la clé de partition a-t-il sur les requêtes ?
- Comment traiter une requête portant sur plusieurs capteurs ?

Scénario D – Dépendances entre équipements

La direction technique souhaite visualiser les relations entre les équipements du campus.

Certains équipements dépendent d'autres installations pour fonctionner.

Exemple :

- Une unité de climatisation dépend d'une alimentation électrique.
- Plusieurs équipements sont raccordés à un même circuit.
- Plusieurs zones dépendent d'une installation centrale.
- Une intervention sur un équipement peut avoir des conséquences sur plusieurs autres équipements.

Paradigme étudié : base orientée graphes.

Travail demandé :

1. Identifiez les différents types de nœuds.
2. Identifiez les relations entre ces nœuds.
3. Déterminez les propriétés à conserver.
4. Proposez trois parcours de recherche pertinents.
5. Expliquez pourquoi une base graphe peut être intéressante pour ces usages.

Exemple de représentation :

```
(Bâtiment)
    |
    | CONTIENT
    |
  (Zone)
    |
    | ACCUEILLE
    |
(Équipement)
```

Questions de réflexion :

- Quels équipements seront impactés par une panne d'un circuit ?
- Quelles zones dépendent d'une installation centrale ?
- Comment retrouver tous les équipements indirectement liés à une installation ?
- Pourquoi une succession de jointures SQL peut-elle devenir complexe pour certains parcours ?

Étape 2 – Comparer les quatre paradigmes

Complétez le tableau suivant.

Critère	Document	Clé-valeur	Wide-Column	Graphe
Structure des données	À compléter	À compléter	À compléter	À compléter

Type d'accès privilégié	À compléter	À compléter	À compléter	À compléter
Avantage principal	À compléter	À compléter	À compléter	À compléter
Limite principale	À compléter	À compléter	À compléter	À compléter
Cas SmartCampus	À compléter	À compléter	À compléter	À compléter
Technologie candidate	À compléter	À compléter	À compléter	À compléter

Vous devez être capables d'expliquer pourquoi chaque paradigme est pertinent pour un scénario et moins adapté à un autre.

Étape 3 – Définir le jeu de données (Dataset)

Un benchmark doit reposer sur un jeu de données représentatif.

Vous devez définir :

- Le nombre de bâtiments.
- Le nombre de zones.
- Le nombre d'équipements.
- Le nombre de capteurs.
- Le nombre de mesures.
- La période couverte par l'historique.
- La fréquence de génération des mesures.

Ces valeurs constituent une hypothèse de travail. Vous pouvez les adapter, à condition de justifier votre choix.

Étape 4 – Construire le protocole de benchmark

Votre protocole doit permettre de comparer les technologies dans des conditions aussi équitables que possible.

1. Définir les scénarios de test

Au minimum deux opérations doivent être comparées :

- Une opération d'écriture ou d'ingestion.
- Une opération de lecture ou de consultation.

Vous pouvez également ajouter une opération d'agrégation.

2. Définir les indicateurs de performance

Indicateur	Définition
Latence p50	Temps de réponse médian
Latence p95	Temps sous lequel 95 % des requêtes répondent
Débit	Nombre d'opérations réalisées par seconde
Taux d'erreur	Part des opérations ayant échoué
CPU	Consommation du processeur
Mémoire	Quantité de mémoire utilisée
Stockage	Volume occupé par les données et index

3. Définir les répétitions

Les tests doivent être répétés plusieurs fois.

Vous devez également prévoir :

- Une phase d'échauffement (warm-up).
- Une phase de mesure.
- Une remise dans un état comparable entre les essais.
- Une conservation des résultats.

L'échauffement permet notamment de limiter l'influence du chargement initial des données et de la mise en cache.

3. Livrables attendus

Dossier

- Analyse des quatre paradigmes NoSQL.
- Schéma de données pour chaque scénario.
- Tableau comparatif des technologies.
- Description du dataset.
- Conclusion sur l'adéquation des technologies aux usages.

Critères de réussite :

Vous devez être capables de relier un besoin concret à un paradigme de stockage, de concevoir une structure adaptée et de définir une comparaison technique reproductible.